

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. void swap(int *a, int *b);
5. void printArray(int *arr, int size);
6.
7. int main()
8. {
9.     int x = 10;
10.    int y = 20;
11.
12.    int *ptr1;
13.
14.    printf("%d\n", *ptr1);
    // ERROR: ptr1 is uninitialized.
    // CORRECTION:
    // int *ptr1 = &x;
    // printf("%d\n", *ptr1);
15.
16.    int *ptr2 = NULL;
17.
18.    *ptr2 = 100;
    // ERROR: Dereferencing a NULL pointer.
    // CORRECTION:
    // ptr2 = malloc(sizeof(int));
    // *ptr2 = 100;
19.
20.    int *ptr3;
21.
22.    ptr3 = &x;
23.
24.    printf("%d\n", *ptr3);
25.
26.    ptr3++;
    // ERROR: Pointer arithmetic moves ptr3 beyond x.
    // CORRECTION: Remove this statement.
27.
28.    printf("%d\n", *ptr3);
    // ERROR: Accessing invalid memory location.
29.
30.    int arr[5] = {10,20,30,40,50};
```

```
31.
32.  int *ptr4 = arr;
33.
34.  printf("%d\n", *(ptr4 + 10));
    // ERROR: Array index out of bounds.
    // CORRECTION: printf("%d\n", *(ptr4 + 4));
35.
36.  int *ptr5;
37.
38.  ptr5 = malloc(sizeof(int));
39.
40.  *ptr5 = 500;
41.
42.  printf("%d\n", *ptr5);
43.
44.  free(ptr5);
45.
46.  printf("%d\n", *ptr5);
    // ERROR: Use-after-free.
    // CORRECTION:
    // ptr5 = NULL;
    // Do not access freed memory.
47.
48.  int *ptr6;
49.
50.  ptr6 = malloc(sizeof(int));
51.
52.  *ptr6 = 1000;
53.
54.  ptr6 = NULL;
    // ERROR: Memory leak. Allocated memory is lost.
    // CORRECTION:
    // free(ptr6);
    // ptr6 = NULL;
55.
56.  int *ptr7;
57.
58.  ptr7 = malloc(5);
    // ERROR: Allocates only 5 bytes.
    // CORRECTION:
    // ptr7 = malloc(5 * sizeof(int));
```

```
59.
60.  for(int i=0; i<10; i++)
    // ERROR: Loop exceeds allocated size.
    // CORRECTION: for(int i=0; i<5; i++)
61.  {
62.      ptr7[i] = i;
63.  }
64.
65.  free(ptr7);
66.
67.  int *ptr8;
68.
69.  ptr8 = malloc(3 * sizeof(int));
70.
71.  ptr8[0] = 10;
72.  ptr8[1] = 20;
73.  ptr8[2] = 30;
74.  ptr8[3] = 40;
    // ERROR: Index 3 is out of bounds.
    // CORRECTION: Allocate 4 integers or remove this line.
75.
76.  free(ptr8);
77.
78.  int *ptr9;
79.
80.  ptr9 = malloc(sizeof(int));
81.
82.  *ptr9 = 50;
83.
84.  free(ptr9);
85.
86.  free(ptr9);
    // ERROR: Double free.
    // CORRECTION:
    // free(ptr9);
    // ptr9 = NULL;
87.
88.  int *ptr10;
89.
90.  ptr10 = malloc(sizeof(int));
91.
```

```

92.  printf("%d\n", *ptr10);
    // ERROR: Uninitialized memory read.
    // CORRECTION:
    // *ptr10 = 0;
    // printf("%d\n", *ptr10);
93.
94.  int *numbers;
95.
96.  numbers = malloc(5 * sizeof(int));
97.
98.  for(int i=0; i<=5; i++)
    // ERROR: Accesses numbers[5], which is invalid.
    // CORRECTION: for(int i=0; i<5; i++)
99.  {
100.     numbers[i] = i * 10;
101.  }
102.
103.  printArray(numbers,5);
104.
105.  int a = 100;
106.  int b = 200;
107.
108.  swap(&a,&b);
109.
110.  printf("%d %d\n", a, b);
    // ERROR: Values remain unchanged because swap() is incorrect.
111.
112.  int **pptr;
113.
114.  int value = 500;
115.
116.  pptr = &value;
    // ERROR: Type mismatch.
    // CORRECTION:
    // int *temp = &value;
    // pptr = &temp;
117.
118.  printf("%d\n", **pptr);
119.
120.  char *name;
121.

```

```

122.  name = malloc(5);
      // ERROR: Insufficient memory allocation.
      // CORRECTION:
      // name = malloc(12);
      // OR allocate strlen("Programming") + 1
123.
124.  strcpy(name,"Programming");
      // ERROR: Buffer overflow.
125.
126.  printf("%s\n", name);
127.
128.  int *ptr11;
129.
130.  ptr11 = malloc(sizeof(int));
131.
132.  *ptr11 = 123;
133.
134.  ptr11++;
      // ERROR: Pointer no longer points to allocated memory.
135.
136.  printf("%d\n", *ptr11);
      // ERROR: Invalid memory access.
137.
138.  int *ptr12 = NULL;
139.
140.  if(*ptr12 > 0)
      // ERROR: Dereferencing NULL pointer.
      // CORRECTION:
      // if(ptr12 != NULL && *ptr12 > 0)
141.  {
142.      printf("Positive\n");
143.  }
144.
145.  int *ptr13;
146.
147.  ptr13 = malloc(100 * sizeof(int));
148.
149.  for(int i=0;i<100;i++)
150.  {
151.      ptr13[i] = i;
152.  }

```

```

153.
154.  ptr13[100] = 500;
      // ERROR: Index 100 is out of bounds.
      // CORRECTION: Use ptr13[99] or allocate 101 elements.
155.
156.  int *ptr14;
157.
158.  ptr14 = malloc(sizeof(int));
159.
160.  free(ptr14);
161.
162.  *ptr14 = 999;
      // ERROR: Use-after-free.
      // CORRECTION:
      // Allocate memory again before use.
163.
164.  int matrix[3][3];
165.
166.  int *p;
167.
168.  p = matrix;
      // WARNING: Incompatible pointer conversion.
      // CORRECTION:
      // int (*p)[3] = matrix;
169.
170.  printf("%d\n", *(p + 20));
      // ERROR: Out-of-bounds access.
171.
172.  int *ptr15;
173.
174.  ptr15 = malloc(sizeof(int));
175.
176.  ptr15 = malloc(sizeof(int));
      // ERROR: Memory leak. Previous allocation lost.
      // CORRECTION:
      // free(ptr15);
      // ptr15 = malloc(sizeof(int));
177.
178.  printf("End of Program\n");
179.
180.  return 0;

```

```
181. }
182.
183. void swap(int *a, int *b)
184. {
185.     int *temp;
186.
187.     temp = a;
188.     a = b;
189.     b = temp;
        // ERROR: Swaps local pointer copies only.
        // CORRECTION:
        // int temp = *a;
        // *a = *b;
        // *b = temp;
190. }
191.
192. void printArray(int *arr, int size)
193. {
194.     for(int i=0; i<=size; i++)
        // ERROR: Accesses one element beyond the array.
        // CORRECTION: for(int i=0; i<size; i++)
195.     {
196.         printf("%d ", arr[i]);
197.     }
198.
199.     printf("\n");
200. }
```